

Jorhat Engineering College

Department of Computer Science and Engineering

An Academic Internship Report

StudyPilot

A Personal AI Study Assistant using RAG and LLMs

Done through the Udemy course

AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents

by Ed Donner (July–August 2026)

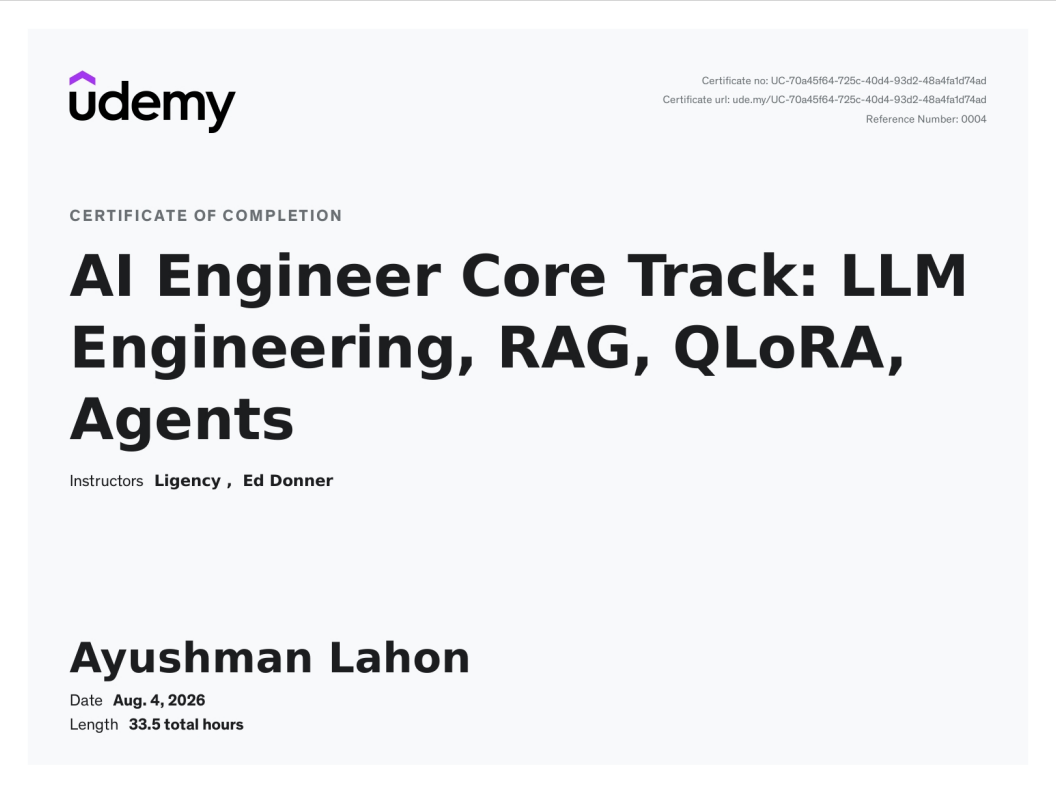
Submitted by

Name	**Ayushman Lahon**
Roll No.	2481102474
Programme	B.Tech, CSE
Semester	5th
Batch	2024–28

Academic Session 2026–27

Course Completion Certificate

I finished the course on **4 August 2026**. The certificate given by Udemy is shown below.


udemy

Certificate no: UC-70a45f64-725c-40d4-93d2-48a4fa1d74ad
Certificate url: ude.my/UC-70a45f64-725c-40d4-93d2-48a4fa1d74ad
Reference Number: 0004

CERTIFICATE OF COMPLETION

AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents

Instructors **Ligency , Ed Donner**

Ayushman Lahon

Date **Aug. 4, 2026**
Length **33.5 total hours**

Certificate No.	UC-70a45f64-725c-40d4-93d2-48a4fa1d74ad
Verify at	https://ude.my/UC-70a45f64-725c-40d4-93d2-48a4fa1d74ad
Course length	33.5 total hours
Instructors	Ligency, Ed Donner
Date of completion	4 August 2026

Declaration

I, **Ayushman Lahon**, a 5th semester student of B.Tech in Computer Science and Engineering (Batch 2024–28) at Jorhat Engineering College, Roll No. 2481102474, declare that this academic internship report titled "StudyPilot: A Personal AI Study Assistant using RAG and LLMs" is a record of my own work. The internship was done in July and August 2026 through the online course "AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents" on Udemy.

The project described here was planned and built by me. This report has not been submitted to any other university or institution for any degree or diploma. Wherever I have used the work of others, I have mentioned it in the references.

Place: Jorhat

Date:

(Ayushman Lahon)
Roll No.: 2481102474

Acknowledgement

I want to thank everyone who helped me during this internship.

First, I thank the **Department of Computer Science and Engineering, Jorhat Engineering College**, for making the academic internship a part of our course and for letting us choose an area we really want to learn.

I am very thankful to **Mr. Ed Donner**, the instructor of "AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents". Every week of his course ends with something that actually works, so I was always building and not only watching. His explanation that an LLM has no memory of its own, and that the program has to send the conversation again every time, changed how I think about chat applications.

I also thank the people who build and maintain the free tools I used, like FastAPI, React, ChromaDB, Ollama, EasyOCR and PyMuPDF, the Qwen team for their open models, and Google for the free tier of the Gemini API. Their documentation helped me a lot when I got stuck.

Lastly, I thank my parents, teachers and friends for their support and patience during this time.

Ayushman Lahon
Roll No.: 2481102474

Abstract

This report covers my academic internship, done in July and August 2026 through the 33.5-hour Udemy course "AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents" by Ed Donner.

The first part of the report is about the course. It taught how to call Large Language Models (LLMs) through their APIs, how to run open-source models on my own computer, how to choose a model for a task, how to build Retrieval-Augmented Generation (RAG) systems with embeddings and a vector database, how to fine-tune models with LoRA and QLoRA, and how several LLM calls and tools can work together as agents.

The second part is about **StudyPilot**, a web application I built using these ideas. A student adds their own study material: lecture PDFs, scanned notes or photos (read with OCR), and YouTube lectures (read from their transcripts). They can then chat with this material and get answers that cite the file and page, revise with flashcards that come back on a spaced-repetition schedule, and take quizzes that are graded with feedback. If a question is not about the uploaded material, the assistant notices this and answers normally instead of forcing an answer from unrelated pages.

The backend uses Python and FastAPI, and the frontend uses React. Chat runs fully on my laptop with the open-source Qwen2.5 1.5B model on a 4 GB RTX 2050 GPU, so the student's notes stay private. Flashcards and quizzes are made and graded by Google Gemini 3.5 Flash-Lite. ChromaDB stores the text chunks and their embeddings, and SQLite stores the flashcards, quizzes and progress.

Keywords: Large Language Models, RAG, Vector Database, OCR, Local LLM, Spaced Repetition, FastAPI, React.

Contents

Course Completion Certificate ii

Declaration iii

Acknowledgement iv

Abstract v

1 Introduction 1
- 1.1 About the Internship . . . 1
- 1.2 The Problem and the Objectives . . . 1

2 What I Learnt in the Course 2
- 2.1 Structure of the Course . . . 2
- 2.2 Weeks 1 and 2: Talking to LLMs through Code . . . 2
- 2.3 Weeks 3 and 4: Open-Source Models and Choosing One . . . 3
- 2.4 Week 5: Retrieval-Augmented Generation (RAG) . . . 3
- 2.5 Weeks 6 and 7: Fine-Tuning . . . 3
- 2.6 Week 8: Agents . . . 3
- 2.7 From the Course to my Project . . . 4

3 Project Design 5
- 3.1 Main Features . . . 5
- 3.2 Architecture . . . 5
- 3.3 API Design . . . 6

4 How the Project Works 7
- 4.1 Adding Study Material . . . 7
- 4.2 Answering a Question . . . 7
- 4.3 Flashcards with Spaced Repetition . . . 8
- 4.4 Quiz and Grading . . . 9

5 Testing and Results 10
- 5.1 Choosing the Relevance Cut-off . . . 10
- 5.2 Test Cases and Speed . . . 10
- 5.3 Problems Found and Fixed . . . 11

6 Conclusion and Future Scope 12
- 6.1 What I Learnt . . . 12
- 6.2 Limitations . . . 12
- 6.3 Conclusion and Future Scope . . . 12

References 13

List of Figures

2.1 How RAG works, in short . . . 3
2.2 The agent loop . . . 4

3.1 Architecture of StudyPilot . . . 5
3.2 FastAPI's automatic test page . . . 6

4.1 Steps for adding study material to the library . . . 7
4.2 Chat: an answer from the library, then a general answer . . . 8
4.3 The Leitner boxes used for flashcards . . . 8
4.4 Flashcards . . . 9
4.5 Taking a quiz and getting feedback . . . 9

List of Tables

1.1 Internship details . . . 1

2.1 Week-wise summary of the course . . . 2
2.2 Course ideas used in the project . . . 4

3.1 Technology stack . . . 6
3.2 Main API routes . . . 6

5.1 Distance of the best chunk for some test questions . . . 10
5.2 Test cases . . . 10

Chapter 1

Introduction

1.1 About the Internship

As a part of the B.Tech curriculum, every student has to complete an academic internship. I did mine in July and August 2026 through a self-paced online course on Udemy. I had used tools like ChatGPT only as a user, and I wanted to learn how LLMs are put inside real software: how they are called from code, how they are given private data, and how they can be made cheaper and more reliable. The details are in Table 1.1.

Table 1.1: Internship details

Item	Details
Type	Academic internship (online course plus own project)
Course	AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents
Platform, instructors	Udemy; Ed Donner (with Ligency)
Length and period	33.5 hours of video, 8 weeks of projects; July–August 2026

1.2 The Problem and the Objectives

Before every exam I face the same problem as most of my classmates. The material for one subject is spread across lecture PDFs, scanned or photographed notes and long recorded lectures on YouTube. When I have a doubt, I do not know which file or which part of a one-hour video has the answer. Making flashcards and practice questions by hand takes so long that I usually skip it. A general chatbot can explain a topic, but it answers from its training, not from *my* syllabus, and it cannot tell me where in my notes something is written. My objectives were:

1. To learn the basics of LLM engineering: model APIs, prompting, open-source models, RAG, fine-tuning and agents.
2. To build a web app that reads PDFs, scanned pages (with OCR) and YouTube lectures into one searchable library, and answers questions from it with the file and page.
3. To help revision with spaced-repetition flashcards and graded quizzes.
4. To run the chat model on my own laptop with a 4 GB GPU, so that notes stay private and there is no running cost.

Chapter 2

What I Learnt in the Course

2.1 Structure of the Course

The course is an eight-week programme. Each week has 3 to 6 hours of video split into "days", and each day has a Jupyter notebook in the course's GitHub repository [2]. Table 2.1 summarises each week, based on the lectures and the notebooks.

Table 2.1: Week-wise summary of the course

Week	Topics	Project
1	Tokens and context windows; the Chat Completions API; system and user prompts; Ollama; stateless calls; JSON output and streaming	Website summariser, company brochure maker
2	OpenAI, Anthropic and Gemini APIs; LiteLLM, LangChain; Gradio; chat with history; tool calling; images and audio	Multi-modal airline support assistant
3	Hugging Face on Colab: pipelines, tokenisers, models, quantisation	Meeting minutes from audio
4	Choosing a model: leaderboards, cost and speed; LLMs for writing code	Python to C++ code converter
5	RAG: chunking, embeddings, Chroma, LangChain, evaluating RAG, reranking and query rewriting	Q&A assistant over a company's documents
6	Data curation; baselines (random forest, XGBoost, neural network); fine-tuning a frontier model with JSONL data	Product price predictor
7	QLoRA fine-tuning of an open-source model on Colab	QLoRA price predictor
8	Agents: Modal deployment, RAG over 400,000 products, specialist, scanner, messenger and planner agents	Multi-agent deal finder (capstone)

2.2 Weeks 1 and 2: Talking to LLMs through Code

An LLM reads *tokens*, small pieces of words, and can only look at a fixed number of them at once (its *context window*). Every call is a list of messages: the *system prompt* gives fixed rules and the *user prompt* gives the task. Two lessons from week 1 went straight into my project. First, the OpenAI way of calling a model became so common that Ollama (for local models) and Google Gemini both offer "OpenAI-compatible" endpoints, so one Python package can talk to all of them. Second, every LLM call is *stateless*: a chat "memory" is only the program sending the whole conversation again. Week 1 also showed JSON output and *streaming*, where words appear as the model writes them. In week 2 we compared the APIs of OpenAI, Anthropic and Gemini, built chat screens with Gradio, and used *tool calling*, where the model asks our code to run a function and then uses the result.

2.3 Weeks 3 and 4: Open-Source Models and Choosing One

Week 3 used open-source models on Hugging Face with a free GPU on Google Colab. I learnt what *quantisation* is: storing the model's numbers with fewer bits so that a bigger model fits in a small GPU. This mattered later, because my laptop GPU has only 4 GB. Week 4 was about choosing a model. Leaderboards help, but in the end you test on your own task and compare cost, speed and quality. I did the same and picked a smaller local model because it was about ten times faster on my laptop.

2.4 Week 5: Retrieval-Augmented Generation (RAG)

This week was the most useful for my project. An *embedding* is a list of numbers that stands for the meaning of a text, so texts with similar meaning get similar numbers. The course used the small model all-MiniLM-L6-v2 (384 numbers per text) and the vector database *Chroma*, which quickly finds the stored texts closest to a question. In RAG (Figure 2.1), documents are cut into *chunks* and stored with their embeddings; for each question, the closest chunks are put into the prompt, so the model answers from those facts instead of from memory. This reduces *hallucination*, where a model confidently says something false.

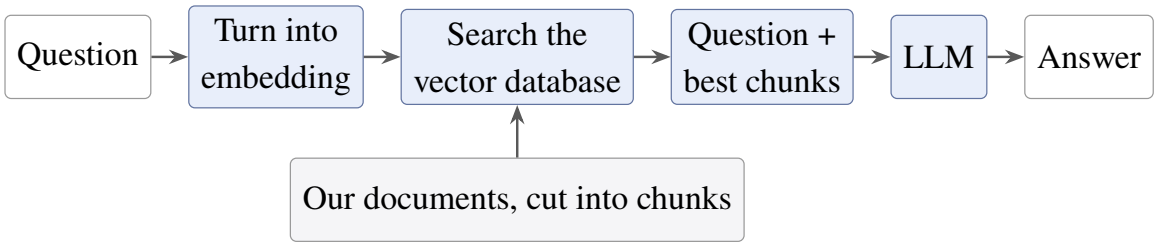


Figure 2.1: How RAG works, in short

The later days covered *evaluating* RAG (check that search finds the right chunks before blaming the model) and advanced ideas like reranking and query rewriting. I used the evaluation habit directly when choosing how strict my search should be (Section 5.1).

2.5 Weeks 6 and 7: Fine-Tuning

Fine-tuning means training an existing model a little more on examples of one task. Weeks 6 and 7 were one project that predicts a product's price from its description. Before any LLM, we cleaned the data and built simple baselines (random forest, XGBoost, a small neural network), then fine-tuned a frontier model with examples in JSONL format. Week 7 used *LoRA*, which keeps the model frozen and trains only small extra matrices, and *QLoRA*, which also stores the frozen model in 4 bits so that an 8-billion-parameter model can be trained on one free Colab GPU. The main lesson: RAG gives a model new *knowledge*, while fine-tuning changes its *behaviour*. That is why StudyPilot uses RAG for the student's notes.

2.6 Week 8: Agents

An *agent* is a loop (Figure 2.2): the model gets a list of tools, decides whether it needs one, our code runs it and sends back the result, and this repeats until the model can answer. The capstone

combined everything: agents that scanned deal websites, estimated prices (including the fine-tuned model running on Modal and RAG over 400,000 products), sent phone notifications, and a planner that decided which to call. StudyPilot does not need a full agent loop, because its steps always happen in the same order, but it uses the same idea of chaining model calls with code checks in between.

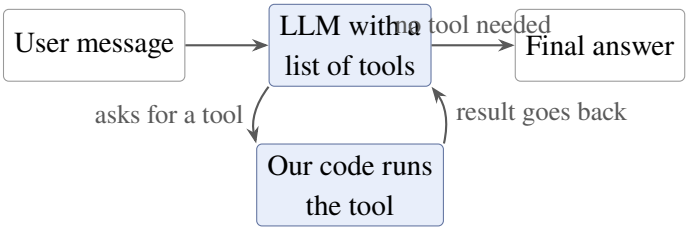


Figure 2.2: The agent loop

2.7 From the Course to my Project

Table 2.2 shows where each idea from the course shows up in StudyPilot. The biggest takeaways for me were: give the model the facts (RAG), remember that the model forgets everything, and measure a change before trusting it.

Table 2.2: Course ideas used in the project

Course idea (week)	**Where it is used in StudyPilot**
OpenAI-compatible APIs (1)	One `openai` package talks to Ollama and to Gemini
System prompts (1, 2)	Separate prompts for answering from notes and for general chat
Stateless calls, memory (1, 2)	The last six messages are sent again with every question
Streaming, JSON output (1)	Chat answers stream word by word; cards and quizzes come back as JSON
Quantisation, model choice (3, 4)	A 4-bit Qwen2.5 1.5B model fits fully in my 4 GB GPU
Embeddings, Chroma, RAG (5)	The whole library is chunked, embedded and searched with Chroma
Evaluating retrieval (5)	Search scores were measured before choosing a relevance cut-off
JSONL data, fine-tuning (6, 7)	A small JSONL dataset for grading short answers (future work)

Chapter 3

Project Design

3.1 Main Features

StudyPilot is a web app for students who want to study from their own material. It has four main parts:

1. **Library.** Upload PDFs, scanned pages or photos, or paste a YouTube link. Everything becomes searchable text.
2. **Chat.** Answers come from the library, with the file and page shown as coloured "citation chips". Small talk and general questions get a normal answer, marked as not coming from the library.
3. **Flashcards.** Decks on any topic in the library, which come back after 1, 2, 4, 8 or 16 days depending on how well each card is known.
4. **Quiz.** Multiple-choice and short-answer questions with a score, feedback on every answer, and a button that turns missed questions into a deck.

3.2 Architecture

Figure 3.1 shows how the parts fit together. The React frontend talks only to my FastAPI backend through a REST API that sends JSON. The backend has an *indexer* that reads files and videos into chunks, a *RAG* part that searches them, a *chat* part that streams answers from the local model, and *flashcards* and *quiz* parts that call Gemini. The original files are kept in a folder, the chunks and embeddings in ChromaDB, and the flashcards, quizzes and progress in SQLite. The notes go to the cloud only when the student makes flashcards or a quiz; chat stays on the laptop. Table 3.1 lists the tools.

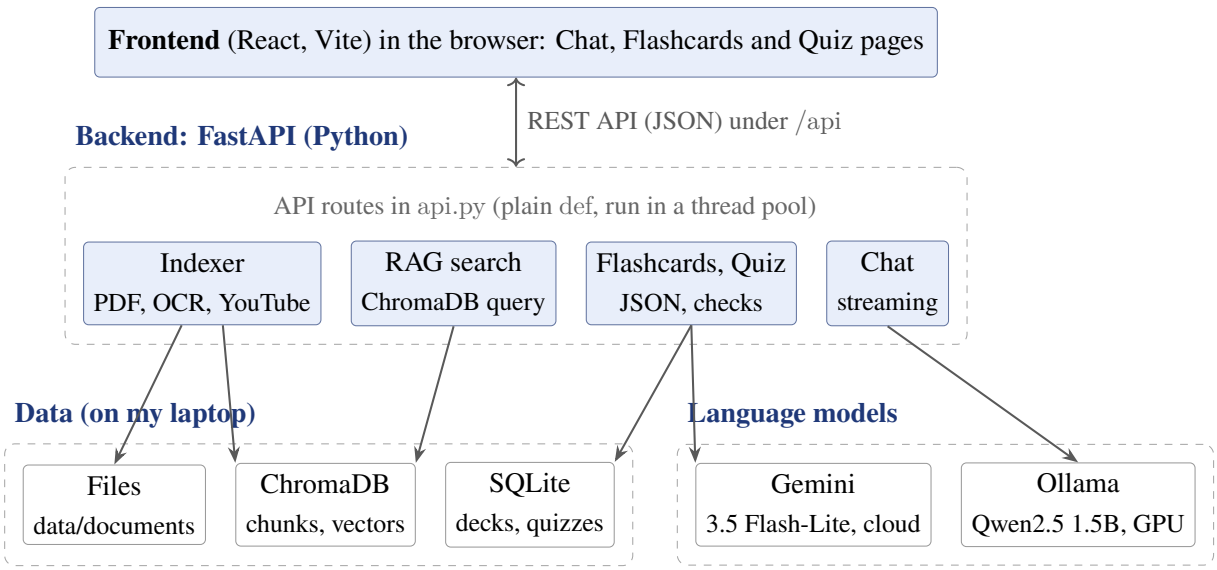


Figure 3.1: Architecture of StudyPilot

Table 3.1: Technology stack

Part	Tool	Why
Backend	Python, FastAPI	Quick to write; checks request data; free test page
Frontend	React, Vite	Pages from small reusable parts; fast reloads
Chat model	Qwen2.5 1.5B on Ollama	Free, private, fits fully in a 4 GB GPU
Study model	Gemini 3.5 Flash-Lite	Free tier; reliable JSON for cards, questions, grades
Search	ChromaDB, all-MiniLM-L6-v2	Runs locally; the same tools as in the course
Database	SQLite	One file, built into Python
Reading input	PyMuPDF, EasyOCR, youtube-transcript-api	PDF text, scans and photos, transcripts

3.3 API Design

The backend is a REST API: the frontend sends a request to a URL with a method (GET to read, POST to create or run an action, DELETE to remove) and gets JSON back. Table 3.2 groups the routes, and Figure 3.2 shows the test page FastAPI builds on its own at /docs, which I used a lot before the frontend was ready. During development, the Vite server forwards every /api request to the backend, so no cross-origin setup is needed.

All routes use plain def, not async def. OCR, embeddings and LLM calls are slow, and FastAPI runs plain def routes in a pool of threads, so a slow upload does not freeze the chat. I tested this by uploading a scanned PDF while asking a question; neither waited for the other.

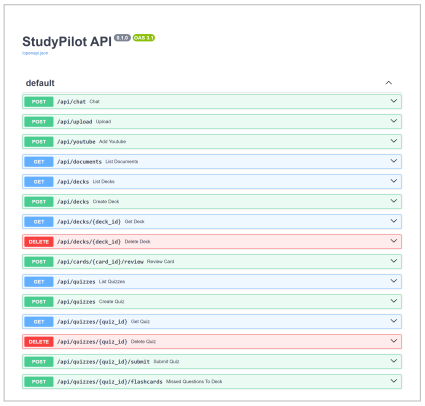


Figure 3.2: FastAPI's automatic test page

Table 3.2: Main API routes

Group	Routes	What they do
Library	POST /api/upload, POST /api/youtube, GET /api/documents	Add material; list it
Chat	POST /api/chat	Answer a question (streamed)
Flashcards	GET, POST /api/decks; GET, DELETE /api/decks/{id}; POST /api/cards/{id}/review	Decks and reviews
Quiz	GET, POST /api/quizzes; GET, DELETE /api/quizzes/{id}; POST .../submit, .../flashcards	Quizzes, grading, missed questions

Chapter 4

How the Project Works

4.1 Adding Study Material

Every input is turned into the same shape: a list of "pages", each with a source name, a page number and its text (Figure 4.1). PDFs are read page by page with PyMuPDF. If a page gives fewer than 50 characters, it is treated as scanned: it is turned into an image at 200 DPI and read with EasyOCR. Photos are read the same way. On a test scan, the OCR text matched 94% of the words of the original. YouTube lectures are read from their transcripts, grouped into 5-minute sections that act like pages; my sharding lecture became 17 sections.

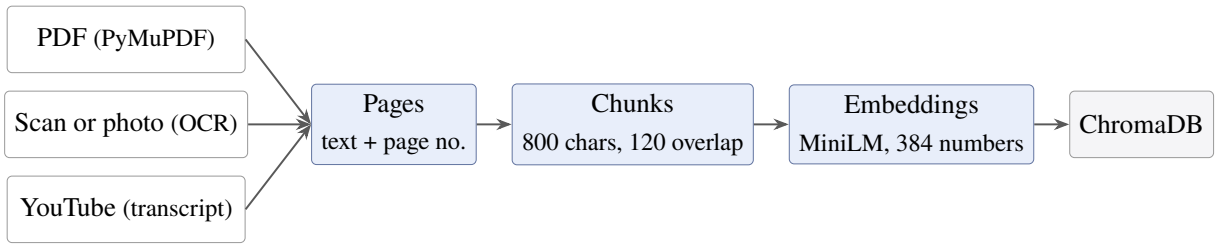


Figure 4.1: Steps for adding study material to the library

The pages are cut into chunks of 800 characters with 120 characters of overlap, so a sentence cut at an edge still appears whole in one chunk. Each chunk is embedded and stored in ChromaDB with its source and page. Uploading the same file again first deletes its old chunks. My test library (two computer-architecture lecture PDFs, a PDF on consistent hashing and a YouTube lecture on database sharding) made 162 chunks.

4.2 Answering a Question

The question is embedded and ChromaDB returns the four closest chunks, each with a *cosine distance* (0 means the same meaning). Chunks with a distance above **0.65** are dropped; this number was measured (Section 5.1). Then:

- **Study mode.** The chunks go into the prompt, each labelled with its source (for example [class16.pdf, page 12]), and the model must answer from them and cite those labels. The browser turns each label into a pink citation chip.
- **General mode.** If no chunk is close enough ("how are you?", "capital of France?"), another system prompt lets the model reply from its own knowledge, and the answer is marked "General answer · not from your library".

My first version always used the four closest chunks, so "how are you" got an answer about database sharding. The cut-off and the two modes fixed this.

Answers are **streamed**: while the model writes, the backend sends one small JSON object per line (first the sources, then pieces of text, then "done"), and the browser adds each piece as it arrives. For **memory**, the browser sends the last six messages with every question, and earlier

answers carry a note of their sources so that follow-ups like "is this from my PDF?" are answered correctly. Small prompt details mattered too: when the excerpts were numbered "Passage 1, 2", the model wrote "Passage 2 says…" in its answers, and a made-up example label in the prompt was once copied as a fake citation. Now the excerpts carry only real source labels.

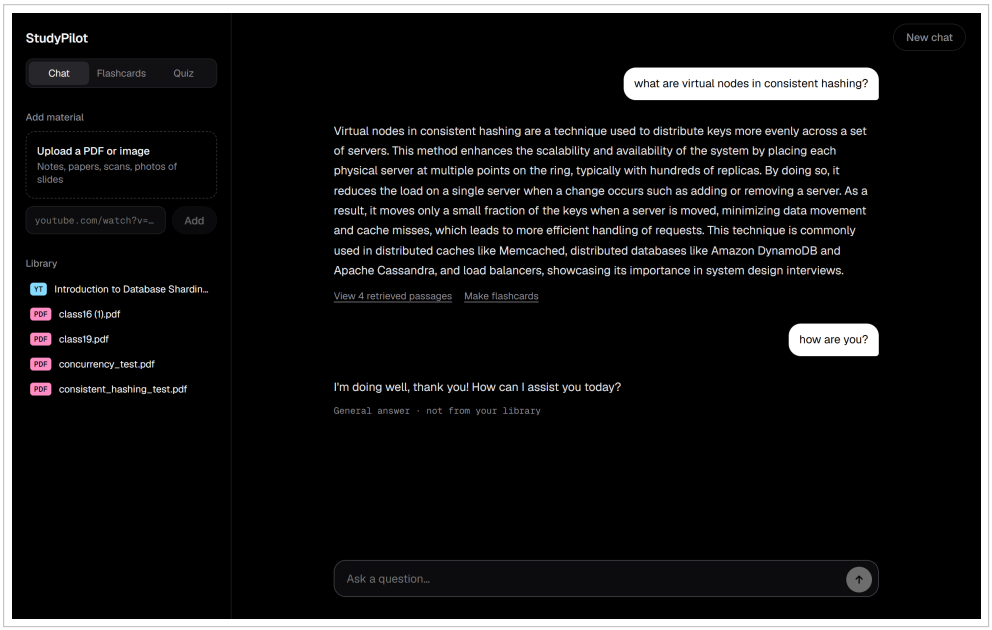


Figure 4.2: Chat: an answer from the library, then a general answer

4.3 Flashcards with Spaced Repetition

For a deck, the student types a topic and picks 5 to 20 cards. The backend retrieves eight chunks on the topic and sends them to Gemini with a JSON schema (front, back, source, page), which forces a reply in exactly that shape. Python keeps only complete cards and removes any source that was not one of the chunks sent, so the model cannot invent a page. Revision uses the *Leitner system* [7] (Figure 4.3): "Got it" moves a card up one box and "Again" sends it back to box 1, and higher boxes come back less often. A card marked "Again" also goes to the back of the current session.

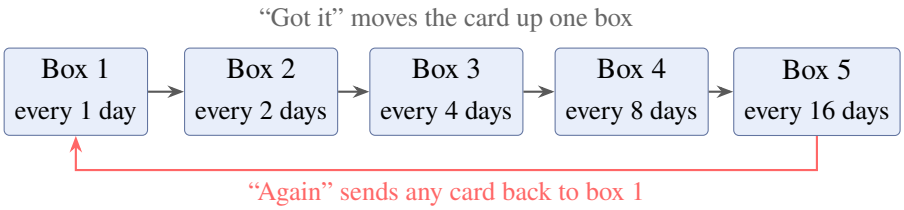
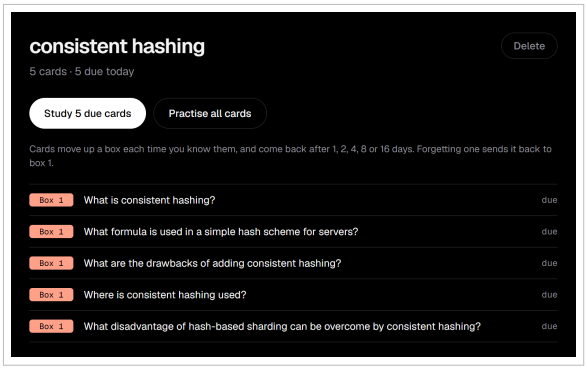
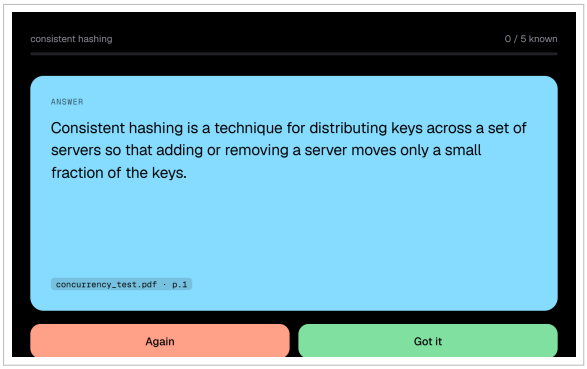


Figure 4.3: The Leitner boxes used for flashcards



(a) A deck with box numbers

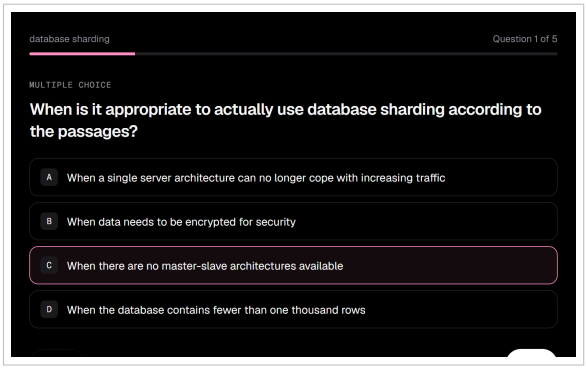


(b) A card turned over, with its source

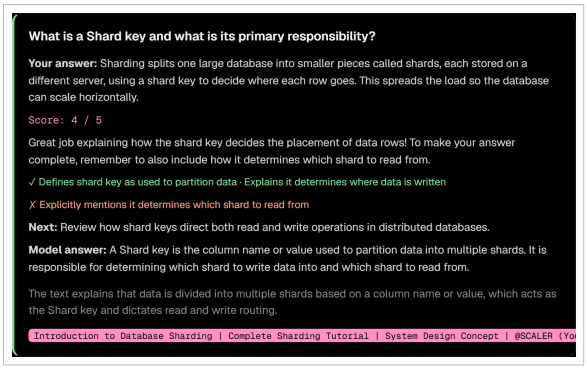
Figure 4.4: Flashcards

4.4 Quiz and Grading

A quiz is made the same way: a topic, 5 to 15 questions, and multiple-choice, short-answer or mixed. Python checks that every multiple-choice question has four options and a valid answer, and shuffles the options because LLMs often put the right one first. The answers stay on the server until the quiz is submitted. **Multiple choice** is graded by plain Python, which is instant and always correct. **Short answers** are graded by Gemini in one request: for each it returns a score from 0 to 5, the key points the student got right, the missing points, short feedback and a next step (Figure 4.5b). This is the same format as my small training dataset for a future fine-tuned grader. Questions that earned less than 60% can be turned into a flashcard deck with one click, without any LLM call.



(a) A multiple-choice question



(b) Feedback on a short answer

Figure 4.5: Taking a quiz and getting feedback

Chapter 5

Testing and Results

5.1 Choosing the Relevance Cut-off

Following the week 5 lesson on evaluating retrieval, I did not guess the cut-off. I asked nine questions about my notes and ten general questions, and noted the distance of the best chunk for each (Table 5.1). Questions about the notes scored 0.13 to 0.61 and general questions scored 0.69 to 0.86, so a cut-off of 0.65 separates them.

Table 5.1: Distance of the best chunk for some test questions

About my notes	Distance	General	Distance
What is consistent hashing?	0.13	Explain recursion	0.69
Instruction level parallelism	0.27	What is photosynthesis?	0.77
How does database sharding work?	0.31	hi	0.79
What is MIMD?	0.57	how are you	0.85
Explain Flynn's taxonomy	0.61	Capital of France?	0.86

5.2 Test Cases and Speed

I tested the API with curl and Python scripts, and the screens with a browser-automation script in Chrome. All the cases in Table 5.2 passed. On my laptop, the sources of an answer appear in about 0.1 s, the first words in about 2.5 s, and a full answer in 3 to 10 s. Making a deck takes about 3 s, a quiz about 4 s, and grading about 2 s.

Table 5.2: Test cases

No.	Test	Result	Status
1	Upload a .txt file	Error 400 with the allowed types	Pass
2	Upload a scanned PDF and a photo	OCR used; 94% of words matched	Pass
3	Add a non-YouTube link	Error 400 with a clear message	Pass
4	Upload and chat at the same time	Both finished, neither waited	Pass
5	Ask "how are you"	Friendly reply, marked general	Pass
6	"Is this from my PDF?" after a cited answer	"Yes", with the right file	Pass
7	Chat while the model is unavailable	Error message shown in the chat	Pass
8	Open a quiz before submitting	No correct answers sent	Pass
9	Submit the same quiz twice	Error 400, "already submitted"	Pass
10	"Got it" twice, then "Again"	Box 1 → 2 → 3 → 1	Pass

5.3 Problems Found and Fixed

The strangest problem was that Gemini requests often failed with an SSL “bad record mac” error and took 10 to 28 seconds when they worked. Testing step by step showed that my network’s IPv6 connection to Google was the cause: over IPv6, 0 of 5 large requests worked, while over IPv4, 5 of 5 worked in about 1.5 seconds. The Gemini client now always uses IPv4. Other fixes included freeing GPU memory after OCR, turning off a progress bar that crashed the OCR model download on Windows, and using the local date (not UTC) for “due today”.

Chapter 6

Conclusion and Future Scope

6.1 What I Learnt

Before this internship I thought building with AI meant writing one clever prompt. I now see it as normal software engineering with one unreliable part in the middle. The model is good at reading and writing, but not at following every rule or knowing where its knowledge came from. So the real work is deciding what the model should do and what plain code should do, giving it the right text (RAG), and checking what comes back. I also learnt to measure instead of guessing: the model, the cut-off and the OCR settings were all chosen by testing.

6.2 Limitations

- The small 1.5B model does not always cite its sources: 5 of 12 test answers had proper citations. The passages used are always listed, though.
- The 0.65 cut-off was measured on my notes; very different subjects may need it measured again. Follow-ups like "tell me more about them" search only the newest question.
- There is no login and no way to delete a document yet, so it suits one student on one computer.
- Flashcards and quizzes depend on Gemini's free tier, which has daily limits.

6.3 Conclusion and Future Scope

The course gave me a clear, hands-on base in LLM engineering, from API calls to RAG, fine-tuning and agents. StudyPilot puts most of these ideas into one working app: it reads PDFs, scanned notes and YouTube lectures into one library, answers from them with citations, knows when a question is not about the notes, and helps revision with spaced-repetition flashcards and graded quizzes, while chat runs fully on a 4 GB laptop GPU. In future, I want to fine-tune a small model with QLoRA on my grading dataset so that quizzes can be graded offline, add query rewriting from week 5 so that follow-ups also search the notes, and add a login and document deletion so that my classmates can use it.

References

[1] Ed Donner and Ligency, *AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents*, online course on Udemy, 2026.

[2] Course code and notebooks: https://github.com/ed-donner/llm_engineering

[3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", 2020. https://arxiv.org/abs/2005.11401

[4] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", 2021, and T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs", 2023. https://arxiv.org

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", 2019. Model used: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[6] Qwen Team, "Qwen2.5 Technical Report", 2024. https://arxiv.org/abs/2412.15115

[7] S. Leitner, *So lernt man lernen* (the Leitner box system for flashcards), 1972.

[8] Documentation of Ollama (https://ollama.com), the Gemini API (https://ai.google.dev), FastAPI (https://fastapi.tiangolo.com), React (https://react.dev), ChromaDB (https://docs.trychroma.com), EasyOCR (https://github.com/JaidedAI/EasyOCR) and PyMuPDF (https://pymupdf.readthedocs.io).